Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică

Laboratory Work 7:

Track A: Three Small Programs: Processes, Redirection, a Pipe

Elaborated: gr. FAF-243
Poiata Calin

Verified:
Patricia Reițman

Chișinău – 2026

Table of Contents

Environment & Toolchain 1
Purpose of the Laboratory Work 1
Part 1. Process Lifecycle: `fork` + `exec` + `wait` 1
1. The Three Steps of the Shell 1
2. Execution and Program Substitution 2
3. Observe Answers for Part 1 2
Part 2. Process Table Lifecycle: A Zombie on Purpose 3
1. Investigating Defunct Processes 3
2. Process Table Inspection 3
3. Observe Answers for Part 2 4
Part 3. I/O Redirection: `ls -l > out.txt` 4
1. File Descriptor Allocation Mechanics 4
2. Execution and File Inspection 4
3. Observe Answers for Part 3 5
Part 4. Inter-Process Pipelines: `ls /etc | wc -l` 5
1. Pipeline Architecture and Implementation 5
2. Execution and Pipeline Diagnostics 6
3. Observe Answers for Part 4 6
Stretch Goal: Multi-Stage Pipeline (`pipeline3.c`) 6
Conclusion 7

Environment & Toolchain

- **Host Operating System:** Linux (Arch Linux x86_64, Kernel 7.2.9-arch1-1)
- **Host Shell:** GNU bash, version 5.3.20(1)-release (`x86_64-pc-linux-gnu`)
- **Host Compiler:** GNU GCC 15.2.1 (`gcc -Wall -Wextra`)
- **Core Introspection Utilities:** `ps` , `head` , `wc` , `ls` , `grep`
- **Document Processor:** Typst 0.15.1 (`typst`)

Purpose of the Laboratory Work

1. Understand the exact lifecycle of a process in Unix: creation through address space replication (`fork`), replacement of process image (`exec`), and parent-child synchronization (`waitpid`).
2. Demystify how a command-line interpreter launches external utilities, noting the distinct stages where the child process identity is formed.
3. Investigate the process table lifecycle and reproduce a defunct/zombie process state on purpose, analyzing how and when process descriptors are reclaimed.
4. Explore the lowest-numbered file descriptor allocation guarantee in POSIX, and demonstrate standard output and standard error redirection without modifying the underlying binaries.
5. Construct inter-process communication pipelines using unnamed unidirectional pipes (`pipe`), duplicating descriptors with `dup` , and evaluating EOF delivery mechanics when file descriptor handles are properly closed versus leaked.
6. Implement and test a multi-stage process pipeline (`pipeline3.c`) connecting three concurrent processes via two distinct pipes.

Part 1. Process Lifecycle: `fork` + `exec` + `wait`

1. The Three Steps of the Shell

When a user types a command such as `ls -l` into a Unix command-line interpreter, the shell executes three fundamental system calls:

1. **Step 1 (`fork`):** Duplicates the calling process. The child process receives a fresh PID and an exact clone of the parent's memory, file descriptor table, and execution state.
2. **Step 2 (`exec`):** Replaces the virtual address space and instruction stream of the child process with the target executable binary (`ls`).
3. **Step 3 (`wait` / `waitpid`):** Suspends parent execution until the child process terminates, retrieving its integer exit status code.

We implemented this architecture in `run.c` :

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:
   ls -l
*/
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }
```

```c
    if (pid == 0) {                             /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                         /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);                   /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

2. Execution and Program Substitution

Compiling and executing `run.c`:

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 49
child:  my PID is 50, my parent is 49
total 112
-rwxrwxrwx 1 calin calin 16376 Nov  2 15:10 pipeline
-rwxrwxrwx 1 calin calin 16376 Nov  2 15:10 pipeline3
-rw-rw-rw- 1 calin calin  1319 Nov  2 15:10 pipeline3.c
-rw-rw-rw- 1 calin calin  1012 Nov  2 15:10 pipeline.c
-rwxrwxrwx 1 calin calin 16416 Nov  2 15:10 redirect
-rw-rw-rw- 1 calin calin   751 Nov  2 15:10 redirect.c
-rwxrwxrwx 1 calin calin 16472 Nov  2 15:10 run
-rw-rw-rw- 1 calin calin   910 Nov  2 15:10 run.c
-rwxrwxrwx 1 calin calin 16528 Nov  2 15:10 zombie
-rw-rw-rw- 1 calin calin   725 Nov  2 15:10 zombie.c
parent: child 50 ended with status 0
```

Replacing `execlp` with `date`:

```
$ ./run_date
parent: my PID is 29
child:  my PID is 30, my parent is 29
Mon Nov  2 03:10:14 PM EET 2026
parent: child 30 ended with status 0
```

Executing a non-existent binary (`nosuchprog`):

```
$ ./run_bad
parent: my PID is 22
child:  my PID is 37, my parent is 22
exec: No such file or directory
parent: child 37 ended with status 127
```

3. Observe Answers for Part 1

1. The line `perror("exec")` is in the child, right after `execlp`. Why is it printed only with `nosuchprog`? On success, `execlp` completely replaces the caller's virtual memory layout (code segment, data, heap, and stack) with the newly loaded program image. Because the original program text no longer exists in memory, instructions following `execlp` never execute. If and only if `execlp` fails (such as when the requested binary does not exist in `$PATH`), it returns `-1` to the caller, allowing execution to continue to `perror("exec")` and `exit(127)`.

2. Which number does the child print as "my parent", and whose PID is it? The child prints `my parent is 49`. This is the Process ID (PID) of the parent process, queried by the child via the `getppid()` system call.

3. What status does the parent see with `ls`, and what status with `nosuchprog`?

- With `ls`, the parent observes exit status **0** (`EXIT_SUCCESS`).
- With `nosuchprog`, the parent observes exit status **127**, which was explicitly passed to `exit(127)` when `exec` failed.

Part 2. Process Table Lifecycle: A Zombie on Purpose

1. Investigating Defunct Processes

When a child process terminates before its parent invokes `wait()` or `waitpid()`, the kernel reclaims its user-space virtual memory, heap allocations, and open file descriptors. However, the process cannot be completely removed from system bookkeeping: the kernel must retain its entry in the process table (`struct task_struct`) containing its PID, termination exit status, and resource usage metrics until the parent collects them. During this interval, the process is in the **Zombie** (or defunct) state.

We created `zombie.c` by inserting `sleep(20);` immediately before `waitpid`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }

    int status;
    sleep(20);                                  /* the new line */
    waitpid(pid, &status, 0);                   /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

2. Process Table Inspection

Starting `zombie` in the background and querying process status during the 20-second sleep:

```
$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie > /dev/null &
$ sleep 2; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
     23       1 Ss+  bash -c cd /tmp/os-lab7 ./zombie > /dev/null & sleep 2 ps -
     24      23 S+   ./zombie
     26      24 Z+   [ls] <defunct>
     27      23 R+   ps -o pid,ppid,stat,cmd
```

Querying again after 20 seconds have elapsed:

```
$ sleep 20; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
     23       1 Rs+  ps -o pid,ppid,stat,cmd
```

3. Observe Answers for Part 2

1. Which letter does `ls` have in the STAT column, and what does it mean? It displays the letter `Z` (with modifier `Z+` indicating a foreground process group). **Meaning:** `Z` signifies a **Zombie** (or defunct) process. The process has completed execution via `exit()`, but its PCB remains in the kernel process table awaiting acknowledgment from its parent process.

2. After 20 seconds, is the `[ls] <defunct>` line still there? Which line of `zombie.c` made it disappear? No, the defunct entry is completely gone. The line `waitpid(pid, &status, 0);` made it disappear. When the parent process terminates its sleep interval and calls `waitpid()`, the kernel transfers the child's exit status to the parent and frees the child's slot in the kernel process table.

Part 3. I/O Redirection: `ls -l > out.txt`

1. File Descriptor Allocation Mechanics

Every Unix process is provisioned with a per-process File Descriptor Table. POSIX specifies a deterministic allocation guarantee: the `open()` and `dup()` system calls **must always allocate the lowest-numbered available index** in the descriptor table.

In `redirect.c`, the child process explicitly closes standard output (`close(1)`). When `open("out.txt", ...)` is called immediately afterward, the kernel assigns descriptor 1 to the newly opened file. When `execlp("ls", ...)` is invoked, `ls` inherits the modified descriptor table and writes its output to descriptor 1 without requiring any code changes.

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        close(1);                                   /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }

    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

2. Execution and File Inspection

```
$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
```

```
done: the listing is in out.txt

$ head -n 5 out.txt
total 112
-rw-r--r-- 1 calin calin     0 Nov  2 15:10 out.txt
-rwxrwxrwx 1 calin calin 16376 Nov  2 15:10 pipeline
-rwxrwxrwx 1 calin calin 16376 Nov  2 15:10 pipeline3
-rw-rw-rw- 1 calin calin  1319 Nov  2 15:10 pipeline3.c
```

3. Observe Answers for Part 3

1. Which number did `open` give, and why exactly that one? `open` returned descriptor **1**. Under POSIX semantics, `open()` must return the lowest-numbered unused descriptor in the calling process's descriptor table. Because `close(1)` freed index 1 (while slots 0 and 2 remained in use), slot 1 was the smallest unallocated integer.

2. `ls` was not changed at all. Why does its listing not appear on the screen? Standard Unix utilities write their formatted output to file descriptor 1 by convention. Programs like `ls` do not examine whether descriptor 1 points to a physical terminal device, a pipe, or a disk file. Because the child bound file descriptor 1 to `out.txt` prior to executing `ls`, all output bytes written via `write(1, ...)` were captured directly into `out.txt`.

3. To redirect the errors instead (like `2> err.txt`), which descriptor would you close before `open`? You would close file descriptor **2** (`close(2);`), which is standard error (`stderr`). The subsequent `open()` call would allocate index 2.

Part 4. Inter-Process Pipelines: `ls /etc | wc -l`

1. Pipeline Architecture and Implementation

A Unix pipeline connects the standard output of one process to the standard input of another via an anonymous in-kernel FIFO buffer created with `pipe(int p[2])`.

- `p[0]` is opened for reading.
- `p[1]` is opened for writing.

The `pipeline.c` implementation manually constructs `ls /etc | wc -l`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                          /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                              /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                          /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);                              /* smallest free number: 0 */
```

```
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);                                    /* the parent uses neither end */
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

2. Execution and Pipeline Diagnostics

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
230

$ ls /etc | wc -l
230
```

When breaking the pipeline by commenting out `close(p[1]);` in the parent process:

```
    close(p[0]);
    /* close(p[1]); */
    wait(NULL);
    wait(NULL);
```

The program hangs indefinitely and must be killed with `Ctrl-C`.

3. Observe Answers for Part 4

1. Did `./pipeline` and `ls /etc | wc -l` give the same number? Yes, both commands produced the exact same line count: **230**.

2. After the change, which of the two children never ends, `ls` or `wc`? `wc` never terminates.

3. Why, in one sentence: what is `wc` waiting for? `wc` is blocked in `read()` waiting for an End-Of-File (EOF) signal on the pipe, which the kernel only generates when all open file descriptors pointing to the write end (`p[1]`) are closed across all processes, but the parent process still holds an open copy of `p[1]`.

Stretch Goal: Multi-Stage Pipeline (`pipeline3.c`)

We extended the architecture to three stages: `ls /usr/bin | grep x | wc -l`. This requires two distinct pipe buffers (`p1` and `p2`) and three concurrent child processes. Every child closes all unused pipe ends after duplicating, and the parent closes all four ends before waiting.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p1[2], p2[2];
    if (pipe(p1) < 0) { perror("pipe 1"); return 1; }
    if (pipe(p2) < 0) { perror("pipe 2"); return 1; }
```

```c
    /* Stage 1: ls /usr/bin → p1[1] */
    if (fork() == 0) {
        close(1);
        dup(p1[1]);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("ls", "ls", "/usr/bin", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    /* Stage 2: grep x (reads p1[0] → writes p2[1]) */
    if (fork() == 0) {
        close(0);
        dup(p1[0]);
        close(1);
        dup(p2[1]);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("grep", "grep", "x", (char *)NULL);
        perror("exec grep");
        exit(127);
    }

    /* Stage 3: wc -l (reads p2[0]) */
    if (fork() == 0) {
        close(0);
        dup(p2[0]);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p1[0]); close(p1[1]);
    close(p2[0]); close(p2[1]);

    wait(NULL);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

Testing against the native shell:

```
$ gcc -Wall -Wextra -o pipeline3 pipeline3.c && ./pipeline3
675

$ ls /usr/bin | grep x | wc -l
675
```

The output confirms exact functional equivalence.

Conclusion

Laboratory Work 7 provided a concrete, empirical examination of process control, file descriptor manipulation, and IPC primitives in the Linux kernel:

1. **Process Lifecycle:** Verified that process creation and program execution are orthogonal operations in Unix (`fork` creates an identical address space, `exec` replaces it, and `wait` reaps termination status).
2. **Defunct/Zombie State:** Demonstrated that a terminated child process remains as a zombie in the process table until reaped by `waitpid()`.

3. **File Descriptor Table Manipulation:** Verified the lowest-free-descriptor rule and showed that I/O redirection operates purely at the file descriptor table level without modifying utility source code.
4. **Pipes and EOF Delivery:** Demonstrated how unidirectional kernel pipe buffers connect concurrent processes and proved that failing to close unused write handles leaves downstream readers permanently blocked waiting for EOF.